UCS503P - Software Engineering Project

CodeSphere AI: An AI-Augmented Software Engineering Collaboration Platform

Grounded Codebase Understanding, Automated Issue Triage, and Team Visibility

Submitted by:

1024030061 Sarthak Soni
1024030059 Ayush Bansal

Group: 3C11 BE Third Year, CSE

Submitted to:

Ms. Paramveer Sidhu

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Higher-Order Goal . . . 3
  - 1.1.2 Time-to-Value Heuristic . . . 3
  - 1.1.3 Problem Statement . . . 3
- 1.2 Project Objectives and Scope . . . 4
  - 1.2.1 Initial Deliverables (Completed for Mid-Semester Evaluation) 4
  - 1.2.2 Subsequent Deliverables (Planned for End-Semester Evaluation) 4

2 Methodology and Design **6**
- 2.1 System Architecture . . . 6
  - 2.1.1 Proposed Solution Overview . . . 6
  - 2.1.2 Core Workflow . . . 6
  - 2.1.3 Operational Constraints for an Educational, Third-Year Setting 7
  - 2.1.4 Technical Stack (Engine Availability Heuristic) . . . 7
- 2.2 Implementation Details . . . 7
  - 2.2.1 Module 1: Local ONNX Codebase Indexing & Grounded RAG Pipeline . . . 8
  - 2.2.2 Module 2: Web Architecture, HMAC Webhook Bot, and Scalability . . . 8

3 Results and Evaluation **10**
- 3.1 Testing and Evaluation Methodology . . . 10
- 3.2 Performance Metrics . . . 10
- 3.3 Analysis: Challenges, Risks, and Mitigations . . . 11

4 Conclusion **12**
- 4.1 Summary of Work . . . 12
- 4.2 Key Findings . . . 12
- 4.3 Future Work and Recommendations (End-Semester Roadmap) . . . 12
- 4.4 Final Remarks . . . 13

Bibliography **14**

Chapter 1

Introduction

1.1 Background and Context

1.1.1 Higher-Order Goal

This project aims to improve software engineering productivity by reducing the time developers spend understanding unfamiliar codebases, triaging issues, and coordinating work across a team. While the underlying theme (AI-assisted developer tooling) is broad, the immediate engineering objective is to translate *faster, better-informed engineering decisions* into a measurable, deployable software solution.

1.1.2 Time-to-Value Heuristic

- We deliver meaningful increments quickly by prototyping the core repository-insight workflow and validating it against a real open-source repository within a short iteration window.
- We prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback.

1.1.3 Problem Statement

Third-year CS teams and early-career developers routinely lose time on tasks that are procedural rather than creative: understanding what a codebase does, finding the right file/module for a bug, writing consistent documentation, and tracking who is doing what. Common pain points include:

- **Onboarding friction:** New contributors take long to understand project structure and module responsibilities.
- **Fragmented task tracking:** Tasks/issues live separately from the code, so context is repeatedly re-explained.
- **Manual, inconsistent code review:** Reviewers spend time on style/documentation nits instead of logic and design.

- **Poor visibility:** Team leads lack a single view of project health (open issues, stale PRs, module ownership).

Why this matters now (contextual relevance): Generative and retrieval-based AI models are mature and cheap enough to embed directly into everyday developer workflows; a lightweight integration layer can meaningfully cut down repetitive engineering overhead even for small teams.

1.2 Project Objectives and Scope

We structure our project objectives into an iteration-friendly scope separating our completed Mid-Semester deliverables from our planned End-Semester roadmap:

1.2.1 Initial Deliverables (Completed for Mid-Semester Evaluation)

1. **GitHub OAuth Login & Repository Connection:** Authenticate users via GitHub OAuth 2.0 and link target repositories.

2. **Local ONNX Indexing & AST Chunking Pipeline:** Chunk source files along syntax boundaries (with minimum 8-line stub merging) and generate 384-dimensional vector embeddings locally in Node.js using `Xenova/all-MiniLM-L6-v2` stored in PostgreSQL `pgvector`.

3. **Grounded AI Assistant Chat with Strict Citation Verification:** Web chat interface with hybrid retrieval, explicit `[CITED: Chunk X]` attribution, negative-query citation suppression, and an automated in-app Grounding Health Probe ($\geq 85\%$ target).

4. **HMAC-Secured GitHub Webhook Auto-Triage Bot:** Automatically post a generated root-cause summary and suggested classification label on newly opened GitHub issues.

5. **CI/CD Pipeline & Staging Deployment:** Automated build verification, unit tests, linting, and continuous deployment to Vercel.

1.2.2 Subsequent Deliverables (Planned for End-Semester Evaluation)

- Multi-pass progressive deep indexing and incremental commit re-indexing for massive repositories.

- Static AST defect and memory leak scanner with automated fork-aware Pull Request generation.
- Team dashboard: live stale-issue alerts (14+ days), commit touch frequency module ownership heatmaps, and task delegation.
- Multi-repository and multi-team collaborative workspace channels.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 Proposed Solution Overview

We build a web-based "CodeSphere AI" dashboard, backed by a GitHub App/webhook bot, that adds AI-assisted codebase understanding on top of GitHub rather than replacing GitHub's own project-management features. We deliberately do *not* build a custom Kanban/task board—GitHub Projects already covers that—and instead surface GitHub's existing issues in our dashboard with an AI layer on top. Core components include:

- **Repository connector:** Link a GitHub repository via OAuth and index its structure, source files, and open issues.
- **AI codebase assistant (website):** Chat interface that answers questions about the codebase ("where is X handled?", "summarize this module") using hybrid retrieval over indexed source files, with clickable citations to exact GitHub file paths and line ranges.
- **Auto-triage bot (GitHub App/webhook):** On every new issue, the bot verifies the HMAC SHA-256 webhook signature and posts a generated summary and suggested label (`bug`, `feature`, `question`, `documentation`, `chore`) as a comment directly on GitHub.
- **Team dashboard (website):** Read-only view of GitHub issues/PRs enriched with staleness alerts and module ownership, pulled live from the GitHub API.

2.1.2 Core Workflow

1. Team lead connects a GitHub repository; the system chunks source files and stores 384-dimensional vector embeddings in PostgreSQL `pgvector`.
2. Contributor asks the AI assistant a question about the codebase (on the website) and receives an answer grounded in retrieved source snippets, with verified file/line references.
3. When a new issue is opened on GitHub, the webhook bot auto-comments with a summary and suggested label within seconds.

4. Team lead views the dashboard for grounding health ($\geq 85\%$ benchmark), backlog health, and module ownership sourced live from GitHub.

2.1.3 Operational Constraints for an Educational, Third-Year Setting

- Keep the solution usable with a single GitHub OAuth connection and no heavyweight infrastructure.
- Avoid dependence on training custom ML models; focus on retrieval, orchestration, and workflow correctness using existing LLM APIs and local ONNX embeddings.
- Design for deployability using common web hosting (Vercel) and managed databases (Supabase PostgreSQL + `pgvector`).

2.1.4 Technical Stack (Engine Availability Heuristic)

A mature set of "engines" (tooling/services) is used to implement this as a web-based project without inventing new infrastructure or training custom models:

- **Repository Access & Auth:** GitHub REST API, OAuth 2.0, and HMAC SHA-256 Webhooks.
- **Framework/Language:** Next.js 14 (App Router), React 18, and Node.js.
- **Database:** Managed PostgreSQL with the `pgvector` extension on Supabase for cosine distance vector search (`<=>`).
- **AI & Local Embeddings:** In-process local ONNX runtime (`@xenova/transformers` running `Xenova/all-MiniLM-L6-v2`) for zero-cost 384-dim embeddings, paired with Google Gemini (`gemini-flash-lite-latest`) for grounded synthesis and triage.
- **Testing & Quality:** Unit/integration test runner, ESLint, and automated in-app Grounding Probe benchmark.
- **CI/CD & Hosting:** GitHub CI workflow and Vercel staging/production deployment.

2.2 Implementation Details

This is an engineering course project, so we focus on scalable integration, retrieval correctness, and workflow automation rather than novel model research.

2.2.1 Module 1: Local ONNX Codebase Indexing & Grounded RAG Pipeline

Code intelligence in CodeSphere AI is strictly retrieval-based, featuring two key engineering enhancements implemented during the Mid-Semester phase:

- **Syntax-Aware AST Boundary Chunking with Stub Merging (`lib/chunk.js`):** Source files are split along function and class declaration boundaries across JavaScript/TypeScript (`export`, `function`, `class`), Go (`func`), Rust (`fn`), and Python (`def`), falling back to 40-line windows (5-line overlap) for non-code files. To prevent 1-line package headers (e.g., `package utils`) from becoming isolated, uninformative chunks, boundaries fewer than 8 lines apart are automatically merged into substantive code blocks.
- **Zero-Cost In-Process Local ONNX Embeddings (`lib/embeddings.js`):** Rather than relying on paid, rate-limited cloud embedding APIs, we execute `Xenova/all-MiniLM-L6-v2` locally inside Node.js via ONNX Runtime. Chunks are embedded in batches of 32 with mean pooling and L_2 normalization, producing 384-dimensional vectors stored in PostgreSQL `pgvector`.
- **Hybrid Retrieval & Strict Citation Defense (`lib/retrieval.js`, `lib/llm.js`):** Queries combine exact file/symbol pattern matching (`ILIKE ANY`) with penalized cosine similarity search (`embedding <=> query_vec`, penalizing non-code templates by $+0.25$ and snippets < 80 chars by $+0.30$). To eliminate hallucinated citations, the LLM must output structured `[CITED: Chunk X]` tags; if a query yields a negative match (e.g., confirming a term is not present anywhere in the repo) or addresses external non-code context, the parser outputs `[CITED: NONE]` and suppresses all source links (sources $= \emptyset$).

2.2.2 Module 2: Web Architecture, HMAC Webhook Bot, and Scalability

Two cooperating services share a unified backend and PostgreSQL database:

- **Website (Frontend + Backend API):** Responsive GitHub-dark UI for the Grounded Copilot, interactive Global Header Search (`/` or `Cmd+K`), and Grounding Health Verification Modal; backend routes handle OAuth, indexing, and retrieval orchestration.
- **GitHub Webhook Service (`/api/webhooks/github`):** Validates incoming payloads using `crypto.timingSafeEqual` against GitHub's `x-hub-signature-256` HMAC SHA-256 digest. On `issues.opened` events, it calls the Gemini orchestration layer (with JSON escape repair) and posts a one-sentence summary and classification label back to the GitHub issue within seconds.

- **Database (`db/schema.sql`):** Stores authenticated users, repository metadata, 384-dimensional chunk embeddings, and persistent Q&A history (`qa_history`) while leaving primary issue state in GitHub.

Scalability Design:

1. **Scalable (Theoretically):** Supports growth in repository size and concurrent teams by decoupling frontend/backend/indexing workers, using stateless REST APIs, and leveraging an `ivfflat` cosine vector index in PostgreSQL.
2. **Operationally Deployable (Practically):** Runs on standard serverless hosting (Vercel) backed by managed Supabase PostgreSQL + `pgvector` and automated CI/CD builds on push.

Chapter 3

Results and Evaluation

3.1 Testing and Evaluation Methodology

We evaluate CodeSphere AI using measurable, attributable metrics tied directly to our core workflow:

- **Primary Evaluation Metric (Answer Grounding Rate):** Percentage of AI-assistant answers that correctly cite an existing, relevant file/line in the indexed repository.
  - *Target:* $\geq 85.0\%$ grounding rate on a held-out test set of architectural and symbol queries.
  - *Automated In-App Verification Probe:* In addition to manual answer-key verification, we built an automated probe endpoint (`/api/repositories/[id]/probe`) that extracts up to 20 real AST symbols from the indexed repository, runs retrieval probes, and computes the exact pass rate in real time.
- **Secondary Evaluation Metrics:**
  - *Time-to-first-understanding:* Time for a new contributor to locate the correct module for a sample task, with vs. without the assistant.
  - *Issue triage accuracy:* Percentage of auto-generated labels/summaries accepted without edit by the contributor.
  - *Triage latency:* Time between an issue being opened on GitHub and the webhook bot posting its summary comment (target: under 30 seconds).
  - *Reliability:* Availability of staging/production for login and repository indexing during business hours (target: $\geq 99\%$ uptime).
- **Pilot Validation Plan:**
  - Recruit 2–3 small student project teams (5–8 contributors total) and index one real repository per team.
  - Compare onboarding/triage metrics before/after within the iteration window by logging baseline estimates via short interviews and manual timing.

3.2 Performance Metrics

Table 3.1 presents the Mid-Semester evaluation results measured on our deployed system:

Evaluation Metric	**Proposal Target**	**Mid-Sem Achieved**
Answer Grounding Rate (Automated Probe)	$\geq 85.0\%$	$95.0\% - 100.0\%$
Negative-Query False Citation Rate	0.0%	0.0%
Webhook Auto-Triage Latency	< 30.0 s	2.8 s
External Embedding API Cost	Low / Quota-bound	\$0.00 (Local ONNX)
Staging / Production Build Health	Pass CI build	100% Passing

Table 3.1: Mid-Semester Evaluation Results Summary

3.3 Analysis: Challenges, Risks, and Mitigations

- **LLM cost and rate limits:** Mitigated by moving vector generation entirely in-process using local ONNX (`all-MiniLM-L6-v2`) at \$0 cost, and using high-quota `gemini-flash-lite-latest` with persisted chat history (`qa_history`) so teammates do not re-run identical queries.
- **Answer and citation hallucination:** Mitigated by enforcing `[CITED: Chunk X]` attribution tags, suppressing citations on negative assertions, and merging < 8-line header stubs during AST chunking.
- **Repository access and privacy:** Mitigated by requesting minimal OAuth scopes and providing full cascade deletion (`DELETE /api/repositories/[id]`) so teams can disconnect and wipe indexed chunks at any time.
- **Evaluation measurement ambiguity:** Mitigated by building the deterministic in-app Grounding Health Probe modal with exportable Markdown audit reports.
- **Operational reliability in pilot:** Mitigated by deploying early to Vercel with automated Next.js production build checks on every push.

Chapter 4

Conclusion

4.1 Summary of Work

In the first half of the semester, we designed, implemented, and deployed the core **CodeSphere AI** platform. We completed all initial deliverables from our proposal—GitHub OAuth login, syntax-aware source chunking, vector indexing in PostgreSQL `pgvector`, grounded codebase chat with clickable GitHub line citations, HMAC-verified webhook issue auto-triage, and production deployment—while additionally introducing zero-cost local ONNX embeddings and strict anti-hallucination citation verification to exceed our $\geq 85\%$ grounding target.

4.2 Key Findings

- **Finding 1 (Hybrid Retrieval + Stub Merging Eliminates Context Noise):** Merging < 8-line header stubs and combining exact filename/symbol matching with penalized cosine similarity lifted grounding accuracy from $\sim 80\%$ to 95%+.
- **Finding 2 (In-Process Local ONNX Removes Embedding Bottlenecks):** Running `all-MiniLM-L6-v2` locally in Node.js delivers fast 384-dimensional vectors without external API keys, billing, or rate limits.
- **Finding 3 (Strict Citation Parsing Prevents False Attribution):** Enforcing `[CITED: Chunk X]` tags and intercepting negative queries ensures 0% false file citations when a queried symbol does not exist in the codebase.

4.3 Future Work and Recommendations (End-Semester Roadmap)

For the End-Semester (EST) evaluation, we will expand the platform with our connected second-phase deliverables:

1. **Progressive Multi-Pass Deep Indexing & Incremental Updates:** Scaling the indexing state machine to process massive multi-hundred-file repositories in resumable micro-batches without serverless timeouts, alongside Just-In-Time (JIT) on-demand file fetching [Tea24].

2. **Static AST Defect Scanner & Fork-Aware Pull Request Engine:** Scanning repositories for non-lintable bugs (memory leaks, unhandled promises, SQL injection risks) and automating end-to-end fix delivery via personal GitHub fork orchestration and upstream Pull Requests.

3. **Team Dashboard & Module Ownership Heatmap:** Surfacing live stale-issue alerts (14+ days) and computing per-directory commit touch frequencies to visualize module ownership.

4. **Multi-Team Collaboration Workspace:** Adding file-level task delegation with deadlines, unread notification alerts, and synchronized team discussion channels.

4.4 Final Remarks

CodeSphere AI fulfills the Mid-Semester UCS503P objectives by delivering a working, deployed, and empirically verified codebase intelligence system. By grounding every AI response in real repository lines and automating initial issue triage, the platform establishes a rock-solid foundation for our End-Semester collaborative and automated PR workflows.

Bibliography

[Tea24] LaTeX Project Team. *LaTeX2e: An updated version of LaTeX*. 2024 (cit. on p. 12).